Singleton Design Pattern

1

2

You may already know but cannot resist to use them in your code

!!! Global Variables!!!

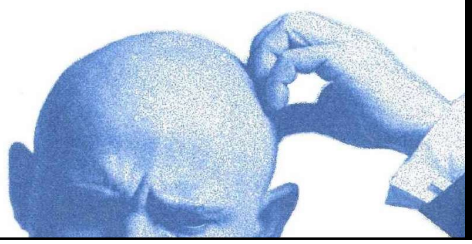
are

EVIL

3

4

It could be worse !!
If you don't know why?



5

Why Global Variables Should Be Avoided When Unnecessary!

- **Non-locality** --總體變數可以被任何地方的程式碼讀寫是難以理解跟除錯的
  Source code is easiest to understand when the scope of its individual elements are limited. Global variables can be read or modified by any part of the program, making it difficult to remember or reason about every possible use.
- **No Access Control or Constraint Checking** – 沒有存取控制以及限制查驗
  A global variable can be get or set by any part of the program, and any rules regarding its use can be easily broken or forgotten. (In other words, get/set accessors are generally preferable over direct data access, and this is even more so for global data.) By extension, the lack of access control greatly hinders achieving security in situations where you may wish to run untrusted code (such as working with 3rd party plugins).

6

Side effects

- **Implicit coupling** – 隱晦的耦合
  A program with many global variables often has tight couplings between some of those variables, and couplings between variables and functions. Grouping coupled items into cohesive units usually leads to better programs.
- **Concurrency issues** - 破壞並型與多執行緒 **thread-safe**
  if globals can be accessed by multiple threads of execution, synchronization is necessary (and too-often neglected). When dynamically linking modules with globals, the composed system might not be thread-safe even if the two independent modules tested in dozens of different contexts were safe.
- **Namespace pollution** - 名稱重疊汙染
  Global names are available everywhere. You may unknowingly end up using a global when you think you are using a local (by misspelling or forgetting to declare the local) or vice versa. Also, if you ever have to link together modules that have the same global variable names, if you are lucky, you will get linking errors. If you are unlucky, the linker will simply treat all uses of the same name as the same object.

7

Side effects

- **Memory allocation issues** – 記憶體配置問題
  Some environments have memory allocation schemes that make allocation of globals tricky. This is *especially* true in languages where "constructors" have side-effects other than allocation (because, in that case, you can express unsafe situations where two globals mutually depend on one another). Also, when dynamically linking modules, it can be unclear whether different libraries have their own instances of globals or whether the globals are shared.
- **Testing and Confinement** – 測試難題
  source that utilizes globals is somewhat more difficult to test because one cannot readily set up a 'clean' environment between runs. More generally, source that utilizes global services of any sort (e.g. reading and writing files or databases) that aren't explicitly provided to that source is difficult to test for the same reason. For communicating systems, the ability to test system invariants may require running more than one 'copy' of a system simultaneously, which is greatly hindered by any use of shared services - including global memory - that are not provided for sharing as part of the test.

8

The stupid way in Java (pure OOPL) Often seen in first-timer OOPers

```
class X {
  ……
};
Main() {
      X  ux = new X();
      ……
      b.dosomthing(ux, …) ;
      // we need object b to do something
      // but it needs cooperation of X
      // and ux is unique ;
}
Class B {
void dosomething(X ux,..) {
     ……
    C.domyjob(ux, …);
    // ok we need c to do something
    // and it needs the cooperation of ux
}
```

In order to let other classes being able to access the unique object, we pass ux all around

9

The Control Class Solution (learned from experience)

```
Class GLOBAL {
static X ux ;
Static void init() {
   ux = new X();
}
}
Somewhere in main() or in any class
   GLOBAL.ux.dosomething();
```

NOTE: you do not new an object from a control class

Make the variable to be static so that it has only one instance which comes with class not object

Class scope

10

OK, is that a problem?

- You assign an object to a global variable
- You call init() to create that object when your application begins
- If the object requires intensive resource and your application never run to the cases which use the object – this is not good.
- We always want an object to be created only when they are needed
- Bad example: You have many image objects and you load them all in the beginning

11

The Singleton Pattern

```
• public class Singleton {
        private static Singleton uniqueInstance ;
        private Singleton() { }
        public static Singleton getInstance() {
                if (uniqueInstance == null) {
                        uniqueInstance = new Singleton();
                }
                return uniqueInstance ;
        }
}
```

12

The keys to singleton

- No public constructor to allow any program to "new" the object
- The new object is created in the getInstance()
- The variable inside the singleton is declared as static

13

How to use a singleton object?

- suppose you have a singleton object called

```
class wife {
  public wife* getInstance();
  void getMoney();
}
```

- to call singleton methods
- in C++

```
wife.getInstance()->getMoney()
```

- in Java or C#

```
wife.getInstance().getMoney()
```

14

Singleton with multithreading

- Sometimes. In a multithreading applications, a singleton will be accessed by more than one thread.
- The synchronization problem arises
-

15

The Singleton Pattern for Multi-thread-Safe – first solution

```
public class Singleton {
        private volatile static Singleton uniqueInstance ;
        private Singleton() { }
        public static synchronized Singleton getInstance() {
                if (uniqueInstance == null) {
                        uniqueInstance = new Singleton();
                }
                return uniqueInstance ;
        }
}
```

16

The Singleton Pattern for Multi-thread-Safe – Double Check lock solution

```
public class Singleton {
        private static Singleton uniqueInstance ;
    private Singleton() { }
        public static  Singleton getInstance() {
                if (uniqueInstance == null) {
                synchronized (Singleton.class) {
                    if (uniqueInstance == null) {
                            uniqueInstance = new Singleton();
                    }
                }
                return uniqueInstance ;
        }
}
```

17

What the hell is "singleton.class"?

- Every Java object created, including every Class loaded, has an associated **lock** or **monitor**. Putting code inside a synchronized block makes the compiler append instructions to **acquire** the lock on the specified object before executing the code, and **release** it afterwards (either because the code finishes normally or abnormally).

18

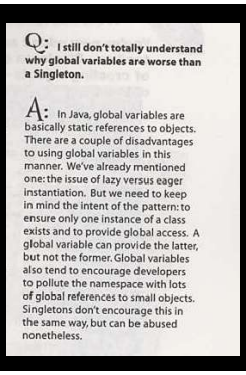
What is a volatile keyword in Java

- It's probably fair to say that on the whole, the volatile keyword in Java is poorly documented, poorly understood, and rarely used. To make matters worse, its formal definition actually changed as of Java 5.
- Essentially, volatile is used to indicate that a **variable's value will be modified by different threads**

19

- Declaring a volatile Java variable means:
  - The value of this variable will **never be cached thread-locally**: all reads and writes will go straight to "main memory";
  - Access to the variable **acts as though it is enclosed in a synchronized block**, synchronized on itself.

20



Q: I still don't totally understand why global variables are worse than a Singleton.

A: In Java, global variables are basically static references to objects. There are a couple of disadvantages to using global variables in this manner. We've already mentioned one: the issue of lazy versus eager instantiation. But we need to keep in mind the intent of the pattern: to ensure only one instance of a class exists and to provide global access. A global variable can provide the latter, but not the former. Global variables also tend to encourage developers to pollute the namespace with lots of global references to small objects. Singletons don't encourage this in the same way, but can be abused nonetheless.

- Q: 一個 singleton，程式碼很多地方也都可以存取啊，你確定跟 global variable 有什麼不一樣?
- A: singleton 可以解決 namespace pollution 等等一些的問題 but still it is just a lazy initialized global variable. 它確保一個獨一無二的 instance 可以存取。

21

Comparisons of Global variables and Singleton

- **The difference**: I would not consider a singleton as a global variable unless the singleton exposed its internal variable through an accessor. Absent accessors, a singleton provides controlled access to the variable, greatly reducing the programming problems.
- **Cohesive Methods** : One of the major arguments against global variables is that they defy the basic OO concept of cohesive methods. With a global variable all of the accesses and operations on the variable are scattered around the code. By pulling all of the methods into a single class or module, they can be evaluated and maintained as a unit.
- ***Should I simply convert a global variable to Singleton to resolve the evil?*** *Converting to the SingletonPattern is common, but you may discover that it makes more sense for the data element to be a member of an existing singleton, or maybe even an instance variable.*

22